CMPSC 448 Midterm Project
Team Leader: Cameryn Lockett
Members: Alyssa Flusser

1. Problem Definition

With the increase in popularity of LLMs, it is increasingly important to find out where text is coming from. The goal of this project is to find out if a classifier can identify the source LLM from response alone (RQ1). This could support transparency in published works.

2. Dataset Curation

Source

We used a subset of TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification. The source file is a benchmark dataset of 66,000 questions and answers across 22 classes, with 3000 samples per class. The dataset was published July 1, 2026.

Subset

We took a random subset of the dataset for three AI sources: ChatGPT, Claude, and Gemini. We kept 400 responses for each, with a total of 1200 examples.

3. Models

CNN

TextCNN uses a 128-dimensional embedding, convolutions with kernel sizes 3, 4, and 5, max-over-time pooling, and dropout 0.5.

RNN

We used 128-dimensional embedding, a bidirectional LSTM with hidden size 128 per direction, dropout 0.5, and a linear output layer over the concatenated final forward and backward states. Sequences are packed with their true lengths so that padding tokens do not affect the final state.

Implementation

Both models are implemented in PyTorch. Responses were lowercase and split into word and punctuation tokens. The vocabulary was built from the training set only, and each response was truncated or padded to 400 tokens. Data were split into 958/122/120 (train/validation/test)

responses, stratified by LLM, with each question kept in one split. CNN and LSTM results are pooled over 3 random seeds.

4. Results

RQ1 — Can we identify which LLM generated a response?

Yes, the LLM that generated a response can be identified from its text alone, although how well depends heavily on the model. We trained a CNN and an LSTM (RNN) to classify 1,200 responses, 400 each from ChatGPT, Claude, and Gemini, using only the response text as input and the source LLM as the label. We split the data into 80/10/10 into train, validation, and test sets, with each question kept within a single split, and we reported results on the held-out test set pooled over 3 random seeds. Both models identify the source LLM above random guessing (33.3%) with the CNN reaching 86.1% accuracy and a macro F1 of 0.862, while the LSTM reached 59.7% accuracy and a macro F1 of 0.596. Both of the models scored above chance, but the CNN is much stronger. The test accuracy per seed for the CNN was [0.867, 0.858, 0.858] and for the LSTM it was [0.592, 0.583, 0.617]. The worst seed for the CNN (0.858) is far above the best seed for the LSTM (0.617).

Both models found Claude the easiest LLM to identify with ChatGPT and Gemini being harder. Most errors came from confusing ChatGPT and Gemini with each other, accounting for 37 of the CNN's 50 mistakes, and 73 of the LSTM's 145 mistakes, suggesting they are the most often confused pair in this sample. A limitation to note for both models is the LLMs' responses differ in length which may cause some of the accuracy to come from length rather than style, which we test in RQ4, where truncating answers to 80 words cost only 4.7 points.

Table 1. Test performance

Model	Accuracy	Macro precision	Macro recall	Macro F1	ChatGPT F1	Claude F1	Gemini F1
CNN	0.861	0.864	0.861	0.862	0.835	0.944	0.807
LSTM (RNN)	0.597	0.608	0.597	0.596	0.505	0.676	0.609
Chance	0.333	-	-	-	-	-	-

Table 2. Confusion matrices (rows=true LLM, columns=predicted LLM)
CNN:

	ChatGPT	Claude	Gemini

ChatGPT	104	1	15
Claude	3	110	7
Gemini	22	2	96

LSTM:

	ChatGPT	Claude	Gemini
ChatGPT	56	17	47
Claude	20	75	25
Gemini	26	10	84

RQ2 — Does the user's prompt help identify the LLM?

We trained the same CNN and LSTM on three kinds of input: the user's question only, the LLM's answer only, and the question followed by the answer.

Input to model	CNN accuracy	CNN macro F1	LSTM accuracy	LSTM macro F1
Question only (input)	0.272	0.268	0.289	0.276
Answer only (output)	0.861	0.862	0.597	0.596
Question + answer	0.828	0.828	0.619	0.618

Chance = 0.333 (3 balanced classes)

i) Can the LLM be predicted from the user's prompt even before seeing its response? ii) Does including the input improve classification?
No, the prompt does not help identify the LLM. Both models score at or slightly below chance (27.2% and 28.9%, against 33.3%), which is consistent with random guessing on a 120-response test set. Neither of the models show a consistent pattern tied to the true LLM. Adding the question to the answer did not improve classification either. It instead lowered the CNN's accuracy from 86.1% to 82.8% and raised the LSTM's from 59.7% to 61.9%. The two changes

point in opposite directions and are small relative to the noise on a 120-response test set, so we read them as having no real effect. Nearly all of the identifying signal comes from the answer.

iii) If input-only classification works surprisingly well, why might this happen?
Question-only classification did not work well. The likely reason is that most questions in our dataset were given to only one source, so the question sets differed across LLMs. The questions look alike across the three LLMs: they average 6.3 to 6.5 words, and about 63% to 65% of each LLM's questions begin with "What". If question-only classification had scored well above chance, the most likely explanation would be that the LLMs were asked different kinds of questions. It would not mean the prompt reveals the LLM.

5. Analysis

RQ3 — Do LLM fingerprints generalize across tasks or domains?

The dataset has no task labels, so we assigned each of the 1,044 unique questions to one of four subject domains: Computing, Engineering & Math; Natural & Life Sciences; Business & Economics; and Humanities & Social Sciences. The labels were LLM-assisted, and we spot-checked a random sample. For each domain, we trained the CNN on the other three and tested on the held-out one, with 3 seeds.

Held-out domain	Test responses	Accuracy	Macro F1	ChatGPT F1	Claude F1	Gemini F1
Computing, Engineering & Math	236	0.811	0.812	0.792	0.859	0.785
Natural & Life Sciences	261	0.866	0.864	0.819	0.954	0.821
Business & Economics	124	0.863	0.864	0.879	0.893	0.820
Humanities & Social Sciences	579	0.845	0.842	0.867	0.879	0.780
For reference: RQ1 (same domains in	360	0.861	0.862	0.835	0.944	0.807

train and test)						

Chance = 0.333. CNN only. Results pooled over 3 seeds. Test sizes differ by domain.

i) Does performance decrease on unseen tasks?
Performance decreases on unseen tasks only slightly. Accuracy on a held-out domain ranged from 81.1% to 86.6%, against 86.1% when the domains were shared (RQ1), and every domain stayed far above the 33.3% chance level. The biggest decrease was Computing, Engineering & Math (81.1%, about 5 points lower). Humanities & Social Sciences dropped about 1.6 points, and Business & Economics and Natural & Life Sciences were within half a point of the RQ1 figure. Most of these differences are within the noise of test sets this small, so the Computing drop is the only one that looks like a real effect.

ii) Are some models more consistently identifiable across domains?
Claude was the easiest to identify in every held-out domain, with F1 between 0.859 and 0.954. Gemini was the hardest in three of the four domains (0.780 to 0.821). In Natural & Life Sciences it was a tie with ChatGPT (0.821 vs 0.819). Gemini's F1 also varied the least (a spread of 0.041, against 0.087 for ChatGPT and 0.095 for Claude), so it was consistently among the hardest to identify, and Claude was consistently the easiest, with the widest spread.

iii) Is the classifier learning a general LLM fingerprint or task-specific patterns?
The results point mostly to a general fingerprint. A classifier that had learned topic-specific wording should have collapsed on a domain it never saw, but accuracy stayed within about 5 points of the in-domain result in all four cases. The largest drop was on the most technical domain, which suggests that a small part of the signal depends on subject vocabulary. Words that appear only in the held-out domain are unknown to the model, which may account for part of the Computing drop.

These domains are subject areas within one question-and-answer dataset, where every response was generated with the same prompting setup. So the results show that the fingerprint carries across subject areas in this kind of answer; they do not show that it would carry to very different tasks such as coding or creative writing. Domain sizes also differ, so the Business & Economics result is the least certain.

RQ4 — What characteristics distinguish different LLMs?

i) What signals does the classifier appear to use?

Claude: it writes the longest answers (298 words, against 240 for ChatGPT and 261 for Gemini). It has the fewest sentences (7.9 vs 10.5 vs 11.1) and the longest ones (39.9 words per sentence vs 23.4 and 28.2). It also uses longer words (6.39 characters on average) and the fewest capitalized words (4.6%).

ChatGPT vs Gemini: these two LLMs look alike in length and sentence structure. Gemini separates from ChatGPT mostly on punctuation. Gemini uses more semicolons and colons (0.005 vs 0.002). ChatGPT uses almost no quotes and apostrophes (0.001), while Gemini and Claude use them far more (0.009 and 0.007). This fits our earlier finding that ChatGPT and Gemini are commonly confused with each other.

Lexical diversity is not a useful signal because the type-token ratio is nearly identical across the three LLMs (ChatGPT 0.626, Claude 0.616, Gemini 0.617), so vocabulary richness does not separate them.

A logistic-regression classifier trained on these hand-built features reaches 0.758 accuracy, and our CNN reaches 0.861. This suggests the CNN relies on these stylistic differences and also on finer phrasing that the 14 features do not capture.

ii) Are the signals linguistic, structural, or both?

The signals are both linguistic and structural. Using only the hand-built features, structural features alone reach 0.692 accuracy, while punctuation alone reaches 0.542 and word-level features alone reach 0.533. All features together reach 0.758, and dropping any one group leaves the accuracy between 0.700 and 0.742, so the groups carry overlapping information.

iii) Does removing certain signals substantially reduce classification performance?

Removing punctuation is the most costly for performance. With punctuation removed, the CNN drops to 0.728 accuracy and 0.730 macro F1, against the 86.1% baseline. Truncating answers to their first 80 words costs 4.7 points, so length is a minor cue. Shuffling word order costs the same 4.7 points. Removing structural groups does not cost much for performance because punctuation and word-level features largely carry the same information.

Condition	Accuracy	Macro F1	Change vs. full text
Full text (RQ1)	0.861	0.862	n/a
First 80 words only	0.814	0.814	-4.7 pts
Punctuation removed	0.728	0.730	-13.3 pts

Word order shuffled	0.814	0.813	-4.7 pts

6. Lessons Learned

From this project, we have learned that it largely is possible to classify LLMs based on their outputs. CNN was able to identify the model 86.1% of the time against 59.7% for the LSTM. Claude was the easiest to identify while ChatGPT and Gemini were most confused with each other. Our results are limited to one dataset with unified prompting, three LLMs, and a small test set of 120 answers, so we cannot say how well the fingerprint holds up on other tasks, other prompts, or other models. We also learned that data preparation is very impactful in shaping results. The data preparation is just as important as making the models themselves. We discovered that testing a few examples before running a full script can save a lot of time.